

Practice Assignment

Transactions · Joins · Indexing · Query Optimization

PART 1

```
CREATE DATABASE TASKS;  
USE TASKS;
```

```
CREATE TABLE customers(  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(50) NOT NULL,  
  email VARCHAR(100) UNIQUE,  
  signup_date DATE  
);
```

```
CREATE TABLE restaurants(  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100) NOT NULL,  
  cuisine VARCHAR(50)  
);
```

```
CREATE TABLE orders(  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  customer_id INT,  
  restaurant_id INT,  
  total DECIMAL(10,2),  
  status VARCHAR(20),  
  created_at DATETIME,
```

```
  FOREIGN KEY(customer_id)  
    REFERENCES customers(id),
```

```
  FOREIGN KEY(restaurant_id)  
    REFERENCES restaurants(id)  
);
```

```
CREATE TABLE accounts(  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(50),  
  balance DECIMAL(10,2)  
);
```

```
INSERT INTO customers(name,email,signup_date) VALUES  
('Siddhant','sidhu@gmail.com','2025-01-10'),
```

```
('Vivaan','vivaan@gmail.com','2025-01-11'),
('Aditya','aditya@gmail.com','2025-01-12'),
('Krishna','krishna@gmail.com','2025-01-13'),
('Rahul','rahul@gmail.com','2025-01-15'),
('Rohan','rohan@gmail.com','2025-01-17'),
('Priya','priya@gmail.com','2025-01-18'),
('Sneha','sneha@gmail.com','2025-01-20'),
('Neha','neha@gmail.com','2025-01-21'),
('Pooja','pooja@gmail.com','2025-01-22'),
('Karan','karan@gmail.com','2025-01-25'),
('Ankit','ankit@gmail.com','2025-01-28'),
('Meera','meera@gmail.com','2025-02-01'),
('Riya','riya@gmail.com','2025-02-03'),
('Ishaan','ishaan@gmail.com','2025-02-04'),
('Harsh','harsh@gmail.com','2025-02-05'),
('Nisha','nisha@gmail.com','2025-02-06'),
('Aisha','aisha@gmail.com','2025-02-07'),
('Jay','jay@gmail.com','2025-02-08'),
('Dhruv','dhruv@gmail.com','2025-02-09');
```

INSERT INTO restaurants(name,cuisine) VALUES

```
('Brothers cafe','Fast Food'),
('Chai G','Indian'),
('Zorko','Fast Food'),
('Sandwichwale','Chinese'),
('Sambharo','Gujarati'),
('Savariya','Gujarati'),
('Vishwakrama','Punjabi'),
('Test of Home','Indian'),
('Better Beans','Cafe'),
('Dosa Express','South Indian');
```

INSERT INTO accounts(name,balance)

VALUES

```
('Alice',6000),
('Bob',3000);
```

INSERT INTO orders(customer_id,restaurant_id,total,status,created_at) VALUES

```
(1,1,450,'completed','2025-03-01 12:00:00'),
(19,2,620,'completed','2025-03-01 13:00:00'),
(3,3,300,'pending','2025-03-01 14:00:00'),
(4,4,550,'completed','2025-03-02 10:00:00'),
(5,5,480,'cancelled','2025-03-02 11:00:00'),
(20,6,390,'completed','2025-03-02 12:30:00'),
(7,7,900,'completed','2025-03-02 13:30:00'),
(8,8,1100,'pending','2025-03-03 14:00:00'),
(9,9,250,'completed','2025-03-03 15:00:00'),
(10,10,420,'completed','2025-03-03 16:00:00'),

(11,1,600,'completed','2025-03-04 12:00:00'),
(12,2,520,'completed','2025-03-04 13:00:00'),
(13,3,275,'pending','2025-03-04 14:00:00'),
```

(14,4,450,'completed','2025-03-05 10:00:00'),
(15,5,720,'completed','2025-03-05 11:00:00'),
(16,6,410,'completed','2025-03-05 12:00:00'),
(18,7,990,'cancelled','2025-03-05 13:00:00'),
(18,8,800,'completed','2025-03-06 14:00:00'),

(1,2,510,'completed','2025-03-06 15:00:00'),
(19,3,330,'completed','2025-03-06 16:00:00'),
(3,4,640,'completed','2025-03-07 10:00:00'),
(4,5,280,'pending','2025-03-07 11:00:00'),
(5,6,350,'completed','2025-03-07 12:00:00'),
(20,7,890,'completed','2025-03-07 13:00:00'),
(7,8,780,'completed','2025-03-07 14:00:00'),
(8,9,210,'completed','2025-03-07 15:00:00'),
(9,10,390,'pending','2025-03-07 16:00:00'),
(10,1,470,'completed','2025-03-08 10:00:00'),

(11,2,550,'completed','2025-03-08 11:00:00'),
(12,3,260,'completed','2025-03-08 12:00:00'),
(13,4,610,'completed','2025-03-08 13:00:00'),
(14,5,700,'completed','2025-03-08 14:00:00'),
(15,6,380,'pending','2025-03-09 10:00:00'),
(16,7,920,'completed','2025-03-09 11:00:00'),
(18,8,830,'completed','2025-03-09 12:00:00'),
(18,9,270,'completed','2025-03-09 13:00:00'),

(1,10,490,'completed','2025-03-10 10:00:00'),
(19,1,520,'cancelled','2025-03-10 11:00:00'),
(3,2,610,'completed','2025-03-10 12:00:00'),
(4,3,320,'completed','2025-03-10 13:00:00'),
(5,4,430,'completed','2025-03-10 14:00:00'),
(20,5,580,'pending','2025-03-10 15:00:00'),
(7,6,350,'completed','2025-03-10 16:00:00'),
(8,7,970,'completed','2025-03-11 10:00:00'),
(9,8,880,'completed','2025-03-11 11:00:00'),
(10,9,230,'completed','2025-03-11 12:00:00'),
(11,10,460,'completed','2025-03-11 13:00:00'),
(12,1,510,'completed','2025-03-11 14:00:00');

PART 2

TASK 1 Transactions (ACID)

a) Write a transaction that transfers ₹200 from one account to the other, but ends in ROLLBACK. Confirm both balances are unchanged afterward.

```
1 START TRANSACTION;
2
3 UPDATE accounts
4 SET balance = balance - 200
5 WHERE id = 1;
6
7 UPDATE accounts
8 SET balance = balance + 200
9 WHERE id = 2;
10
11 ROLLBACK;
12
13 SELECT * FROM accounts;
```

id	name	balance
1	Alice	6000.00
2	Bob	3000.00

#	Time	Action	Message	Duration / Fetch
48	19:39:55	UPDATE accounts SET balance = balance - 200 WHERE id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
49	19:40:00	UPDATE accounts SET balance = balance + 200 WHERE id = 2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
50	19:40:04	ROLLBACK	0 row(s) affected	0.015 sec
51	19:40:08	SELECT * FROM accounts LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 s

b) Write the same transfer again, this time ending in COMMIT. Confirm both balances have changed.

```
15 START TRANSACTION;
16
17 UPDATE accounts
18 SET balance = balance - 200
19 WHERE id = 1;
20
21 UPDATE accounts
22 SET balance = balance + 200
23 WHERE id = 2;
24
25 COMMIT;
26 SELECT * FROM accounts;
27
```

id	name	balance
1	Alice	5800.00
2	Bob	3200.00

#	Time	Action	Message	Duration / Fetch
53	19:42:28	UPDATE accounts SET balance = balance - 200 WHERE id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
54	19:42:32	UPDATE accounts SET balance = balance + 200 WHERE id = 2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
55	19:42:36	COMMIT	0 row(s) affected	0.016 sec
56	19:42:41	SELECT * FROM accounts LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 s

c) In 2–3 sentences, explain which ACID property guarantees the money is never "lost" in the middle of a transfer.

→ Acid property's Atomicity property guarantees the money is never "lost" in the middle of a transfer because it says that either it done 100% or 0% like in rollback it is 0% and in commit case it 100% done.

TASK 2 Joins

- a) Write an INNER JOIN that lists every customer who has placed at least one order.

The screenshot shows a SQL IDE with a query window and a results grid. The query is an INNER JOIN between customers and orders.

```
1 SELECT c.id, c.name, c.email, o.id AS order_id, o.total, o.status
2 FROM customers c
3 INNER JOIN orders o
4 ON c.id = o.customer_id;
```

The results grid displays 16 rows of data:

id	name	email	order_id	total	status
1	Siddhant	sidhu@gmail.com	1	450.00	completed
19	Jay	jay@gmail.com	2	620.00	completed
3	Aditya	aditya@gmail.com	3	300.00	pending
4	Krishna	krishna@gmail.com	4	550.00	completed
5	Rahul	rahul@gmail.com	5	480.00	cancelled
20	Dhruv	dhruv@gmail.com	6	390.00	completed
7	Priya	priya@gmail.com	7	900.00	completed
8	Sneha	sneha@gmail.com	8	1100.00	pending
9	Neha	neha@gmail.com	9	250.00	completed
10	Pooja	pooja@gmail.com	10	420.00	completed
11	Karan	karan@gmail.com	11	600.00	completed
12	Ankit	ankit@gmail.com	12	520.00	completed
13	Meera	meera@gmail.com	13	275.00	pending
14	Riya	riya@gmail.com	14	450.00	completed
15	Ishaan	ishaan@gmail.com	15	720.00	completed
16	Harsh	harsh@gmail.com	16	410.00	completed

The output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
55	19:42:36	COMMIT	0 row(s) affected	0.016 sec
56	19:42:41	SELECT * FROM accounts LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000
57	19:46:03	SELECT c.id, c.name, c.email, o.id AS order_id, o.total, o.status FROM customers c INNER JOIN orders o ON ...	48 row(s) returned	0.000 sec / 0.000

- b) Write a LEFT JOIN that lists every customer, including those with zero orders. Then add a WHERE clause to isolate only the customers who have never ordered.

The screenshot shows a SQL IDE with a query window and a results grid. The query is a LEFT JOIN between customers and orders, filtered for customers who have never ordered.

```
5
6 SELECT c.id, c.name, c.email, o.id AS order_id
7 FROM customers c
8 LEFT JOIN orders o
9 ON c.id = o.customer_id;
10
```

The results grid displays 26 rows of data, including customers with no orders (order_id is NULL):

id	name	email	order_id
1	Siddhant	sidhu@gmail.com	1
1	Siddhant	sidhu@gmail.com	19
1	Siddhant	sidhu@gmail.com	37
2	Vivaan	vivaan@gmail.com	1000
3	Aditya	aditya@gmail.com	3
3	Aditya	aditya@gmail.com	21
3	Aditya	aditya@gmail.com	39
4	Krishna	krishna@gmail.com	4
4	Krishna	krishna@gmail.com	22
4	Krishna	krishna@gmail.com	40
5	Rahul	rahul@gmail.com	5
5	Rahul	rahul@gmail.com	23
5	Rahul	rahul@gmail.com	41
6	Rohan	rohan@gmail.com	1000
7	Priya	priya@gmail.com	7
7	Priya	priya@gmail.com	25
7	Priya	priya@gmail.com	43
8	Sneha	sneha@gmail.com	8
8	Sneha	sneha@gmail.com	26

The output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
56	19:42:41	SELECT * FROM accounts LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000
57	19:46:03	SELECT c.id, c.name, c.email, o.id AS order_id, o.total, o.status FROM customers c INNER JOIN orders o ON ...	48 row(s) returned	0.000 sec / 0.000
58	19:50:54	SELECT c.id, c.name, c.email, o.id AS order_id FROM customers c LEFT JOIN orders o ON c.id = o.customer_id;	51 row(s) returned	0.000 sec / 0.000

- c) In one sentence, explain why a marketing team would prefer the result of query (b) over query (a).
→ A marketing team would prefer the result of query (b) because it identifies customers who have never placed an order, and this allow them to whom to give an offer and discounts.

TASK 3 Indexing

- a) Pick a column that is not currently indexed (for example status or created_at).
→ Here I picked column status which is not indexed.
- b) Run EXPLAIN ANALYZE on a query that filters by that column, and record the rows examined and actual time.

The screenshot shows a SQL IDE with a query editor and a result grid. The query is:

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM orders
4 WHERE status='completed';
5
6
```

The result grid shows the execution plan for the query:

```
EXPLAIN
-> Filter: (orders."status" = 'completed') (cost=5.05 rows=4.8) (actual time=0.0638..0.102 rows=38 loops=1) -> Table scan on orders (cost=5.05 rows=48) (actual time=0.0605..0.0846 rows=48 loops=1)
```

Below the result grid, there is an 'Output' section showing the execution of the query and the creation of an index:

#	Time	Action	Message	Duration / Fetch
59	19:53:32	EXPLAIN ANALYZE SELECT * FROM orders WHERE status='completed'	1 row(s) returned	0.000 sec / 0.00
60	19:54:29	DROP INDEX idx_status ON orders	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.015 sec
61	19:54:38	EXPLAIN ANALYZE SELECT * FROM orders WHERE status='completed'	1 row(s) returned	0.000 sec / 0.00

- c) Create an index on that column.

```
7 CREATE INDEX idx_status
8 ON orders(status);
9
10
```

The screenshot shows the 'Output' section of a SQL IDE, displaying the execution of a query and the creation of an index:

#	Time	Action	Message	Duration / Fetch
60	19:54:29	DROP INDEX idx_status ON orders	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.015 sec
61	19:54:38	EXPLAIN ANALYZE SELECT * FROM orders WHERE status='completed'	1 row(s) returned	0.000 sec / 0.000 se
62	19:55:25	CREATE INDEX idx_status ON orders(status)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec

d) Re-run the exact same query and record the new numbers.

The screenshot shows a SQL IDE interface with a query editor at the top and a results pane at the bottom. The query editor contains the following SQL code:

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM orders
4 WHERE status='completed';
5
6
```

The results pane shows the output of the query, which is an EXPLAIN statement. The output is as follows:

```
EXPLAIN
-> Index lookup on orders using idx_status (status='completed') (cost=4.55 rows=38) (actual time=0.24..0.264 rows=38 loops=1)
```

Below the results pane, there is a table showing the execution plan. The table has four columns: #, Time, Action, and Message. The data is as follows:

#	Time	Action	Message
61	19:54:38	EXPLAIN ANALYZE SELECT * FROM orders WHERE status='completed'	1 row(s) returned
62	19:55:25	CREATE INDEX idx_status ON orders(status)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
63	19:56:03	EXPLAIN ANALYZE SELECT * FROM orders WHERE status='completed'	1 row(s) returned

e) Submit a one-line comparison of before vs. after.

→ Before creating the index, MySQL scanned the entire table; after creating the index, it used the `idx_status` index, reducing the number of rows examined and improving query performance.

TASK 4 Query Optimization

Start with this query, rewritten to use your own table and column names:

```
SELECT * FROM orders WHERE customer_id IN (  
  SELECT id FROM customers WHERE signup_date >= 'some date'  
);
```

a) Rewrite it as a JOIN instead of a subquery.

The screenshot shows a SQL IDE with a query window and a results grid. The query is:

```
SELECT *  
FROM orders o  
INNER JOIN customers c  
ON o.customer_id = c.id  
WHERE c.signup_date >= '2025-02-01';
```

The results grid displays 18 rows of data. Below the grid, the 'Action Output' pane shows the execution details for three actions:

#	Time	Action	Message	Duration / Fetch
62	19:55:25	CREATE INDEX idx_status ON orders(status)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
63	19:56:03	EXPLAIN ANALYZE SELECT * FROM orders WHERE status='completed'	1 row(s) returned	0.000 sec / 0.000
64	19:58:50	SELECT * FROM orders o INNER JOIN customers c ON o.customer_id = c.id WHERE c.signup_date >= '2025-02-01'	18 row(s) returned	0.000 sec / 0.000

b) Rewrite it again to select only the specific columns you'd actually need on a screen (and avoid SELECT *).

The screenshot shows a SQL IDE with a query window. The query is:

```
SELECT  
  o.id AS order_id,  
  c.name,  
  c.email,  
  o.total,  
  o.status,  
  o.created_at  
FROM orders o  
INNER JOIN customers c  
ON o.customer_id = c.id  
WHERE c.signup_date >= '2025-02-01';
```

order_id	name	email	total	status	created_at
13	Meera	meera@gmail.com	275.00	pending	2025-03-04 14:00:00
31	Meera	meera@gmail.com	610.00	completed	2025-03-08 13:00:00
14	Riya	riya@gmail.com	450.00	completed	2025-03-05 10:00:00
32	Riya	riya@gmail.com	700.00	completed	2025-03-08 14:00:00
15	Ishaan	ishaan@gmail.com	720.00	completed	2025-03-05 11:00:00
33	Ishaan	ishaan@gmail.com	380.00	pending	2025-03-09 10:00:00
16	Harsh	harsh@gmail.com	410.00	completed	2025-03-05 12:00:00
34	Harsh	harsh@gmail.com	920.00	completed	2025-03-09 11:00:00
17	Aisha	aisha@gmail.com	990.00	cancelled	2025-03-05 13:00:00
18	Aisha	aisha@gmail.com	800.00	completed	2025-03-06 14:00:00
35	Aisha	aisha@gmail.com	830.00	completed	2025-03-09 12:00:00
36	Aisha	aisha@gmail.com	270.00	completed	2025-03-09 13:00:00
2	Jay	jay@gmail.com	620.00	completed	2025-03-01 13:00:00
20	Jay	jay@gmail.com	330.00	completed	2025-03-06 16:00:00
38	Jay	jay@gmail.com	520.00	cancelled	2025-03-10 11:00:00
6	Dhruv	dhruv@gmail.com	390.00	completed	2025-03-02 12:30:00
24	Dhruv	dhruv@gmail.com	890.00	completed	2025-03-07 13:00:00
42	Dhruv	dhruv@gmail.com	580.00	pending	2025-03-10 15:00:00

#	Time	Action	Message	Duration / Fetch
63	19:56:03	EXPLAIN ANALYZE SELECT * FROM orders WHERE status='completed'	1 row(s) returned	0.000 sec / 0.000
64	19:58:50	SELECT * FROM orders o INNER JOIN customers c ON o.customer_id = c.id WHERE c.signup_date >= '2025-02-01'	18 row(s) returned	0.000 sec / 0.000
65	20:00:00	SELECT o.id AS order_id, c.name, c.email, o.total, o.status, o.created_at FROM orders o INNER JOIN customers c ON o.customer_id = c.id	18 row(s) returned	0.000 sec / 0.000

c) Run EXPLAIN ANALYZE on your original query and your final rewritten version, and note what changed.

order_id	name	email	total	status	created_at
13	Meera	meera@gmail.com	275.00	pending	2025-03-04 14:00:00
31	Meera	meera@gmail.com	610.00	completed	2025-03-08 13:00:00
14	Riya	riya@gmail.com	450.00	completed	2025-03-05 10:00:00
32	Riya	riya@gmail.com	700.00	completed	2025-03-08 14:00:00
15	Ishaan	ishaan@gmail.com	720.00	completed	2025-03-05 11:00:00
33	Ishaan	ishaan@gmail.com	380.00	pending	2025-03-09 10:00:00
16	Harsh	harsh@gmail.com	410.00	completed	2025-03-05 12:00:00
34	Harsh	harsh@gmail.com	920.00	completed	2025-03-09 11:00:00
17	Aisha	aisha@gmail.com	990.00	cancelled	2025-03-05 13:00:00
18	Aisha	aisha@gmail.com	800.00	completed	2025-03-06 14:00:00
35	Aisha	aisha@gmail.com	830.00	completed	2025-03-09 12:00:00
36	Aisha	aisha@gmail.com	270.00	completed	2025-03-09 13:00:00
2	Jay	jay@gmail.com	620.00	completed	2025-03-01 13:00:00
20	Jay	jay@gmail.com	330.00	completed	2025-03-06 16:00:00
38	Jay	jay@gmail.com	520.00	cancelled	2025-03-10 11:00:00
6	Dhruv	dhruv@gmail.com	390.00	completed	2025-03-02 12:30:00
24	Dhruv	dhruv@gmail.com	890.00	completed	2025-03-07 13:00:00
42	Dhruv	dhruv@gmail.com	580.00	pending	2025-03-10 15:00:00


```

28
29
30 • EXPLAIN ANALYZE
31 SELECT *
32 FROM orders
33 WHERE customer_id IN (
34     SELECT id
35     FROM customers
36     WHERE signup_date >= '2025-02-01'
37 );
38

```


#	Time	Action	Message	Duration / Fetch
64	19:58:50	SELECT * FROM orders o INNER JOIN customers c ON o.customer_id = c.id WHERE c.signup_date >= '2025-02-01'	18 row(s) returned	0.000 sec / 0.000
65	20:00:00	SELECT o.id AS order_id, c.name, c.email, o.total, o.status, o.created_at FROM orders o INNER JOIN customers c ON o.customer_id = c.id	18 row(s) returned	0.000 sec / 0.000
66	20:01:08	EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id IN (SELECT id FROM customers WHERE signup_date >= '2025-02-01')	1 row(s) returned	0.000 sec / 0.000

Query 1
SQL File 3*
SQL File 4*
SQL File 5*
SQL File 6* x

Limit to 1000 rows

```

37  );
38
39  • EXPLAIN ANALYZE
40  SELECT
41      o.id AS order_id,
42      c.name,
43      c.email,
44      o.total,
45      o.status,
46      o.created_at
47  FROM orders o
48  INNER JOIN customers c
49  ON o.customer_id = c.id
50  WHERE c.signup_date >= '2025-02-01';
51

```

Result Grid
Filter Rows:
Exports:
Wrap Cell Content:

EXPLAIN
-> Nested loop inner join (cost=8.84 rows=18.8) (actual time=0.0766..0.152 rows=18 loops=1) -> Filter: (c.signup_date >= DATE'2025-02-01') (cost=2.25 rows=6.67) (actual time=0.0489..0.0589 rows=8 loops=1) -> Table scan on c (cost...

Result 9 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
65	20:00:00	SELECT o.id AS order_id, c.name, c.email, o.total, o.status, o.created_at FROM orders o INNE...	18 row(s) returned	0.000 sec / 0.000
66	20:01:08	EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id IN (SELECT id FROM customers ...	1 row(s) returned	0.000 sec / 0.000
67	20:02:01	EXPLAIN ANALYZE SELECT o.id AS order_id, c.name, c.email, o.total, o.status, o.created_at ...	1 row(s) returned	0.000 sec / 0.000

PART 3

TASK 5 The Customer Profile Page

- a) Write a query that joins orders with restaurants (and customers, if needed) to return: restaurant name, order total, order status, and created_at — filtered to one specific customer, sorted by the most recent order first, limited to 5 results.

The screenshot shows a database IDE with a SQL query in the editor and its results in a table below. The query is as follows:

```
1 • USE tasks;
2
3
4 • SELECT *
5 FROM orders o
6 INNER JOIN restaurants r
7 ON o.restaurant_id = r.id
8 WHERE o.customer_id = 1
9 ORDER BY o.created_at DESC
10 LIMIT 5;
```

The results table shows 5 rows of data:

	id	customer_id	restaurant_id	total	status	created_at	id	name	cuisine
▶	37	1	10	490.00	completed	2025-03-10 10:00:00	10	Dosa Express	South Indian
	19	1	2	510.00	completed	2025-03-06 15:00:00	2	Chai G	Indian
	1	1	1	450.00	completed	2025-03-01 12:00:00	1	Brothers cafe	Fast Food

Below the table, the 'Action Output' section shows the execution of the query:

#	Time	Action	Message	Duration / Fetc
1	16:54:36	USE tasks	0 row(s) affected	0.000 sec
2	16:54:42	SELECT * FROM orders o INNER JOIN restaurants r ON o.restaurant_id = r.id WHERE o.customer_id = 1 ORDE...	3 row(s) returned	0.015 sec / 0.1

- b) Run EXPLAIN ANALYZE on it and record the rows examined and time taken.

The screenshot shows the same database IDE with the query modified to include EXPLAIN ANALYZE:

```
19 • EXPLAIN ANALYZE
20 SELECT *
21 FROM orders o
22 INNER JOIN restaurants r
23 ON o.restaurant_id = r.id
24 WHERE o.customer_id = 1
25 ORDER BY o.created_at DESC
26 LIMIT 5;
```

The 'Action Output' section shows the execution of the query with EXPLAIN ANALYZE:

#	Time	Action	Message	Duration / Fetc
1	17:00:10	EXPLAIN ANALYZE SELECT * FROM orders o INNER JOIN restaurants r ON o.restaurant_id = r.id WHERE o.c...	Error Code: 1046. No database selected Select the default DB to be used by double-clicking its name in the SCH...	0.016 sec
2	17:00:18	USE tasks	0 row(s) affected	0.000 sec
3	17:00:23	EXPLAIN ANALYZE SELECT * FROM orders o INNER JOIN restaurants r ON o.restaurant_id = r.id WHERE o.c...	1 row(s) returned	0.016 sec / 0.1

- c) Create the index (or indexes) you think will make this query fast. Think about which column(s) appear in your WHERE clause and your ORDER BY.

The screenshot shows a database IDE with a SQL editor and an output window. The SQL editor contains the following code:

```
31
32 • CREATE INDEX idx_customer_created
33 ON orders(customer_id, created_at);
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
```

The output window shows the following actions and messages:

#	Time	Action	Message	Duration / Fetch
1	17:00:10	EXPLAIN ANALYZE SELECT * FROM orders o INNER JOIN restaurants r ON o.restaurant_id = r.id WHERE o.c...	Error Code: 1046. No database selected Select the default DB to be used by double-clicking its name in the SCH...	0.016 sec
2	17:00:18	USE tasks	0 row(s) affected	0.000 sec
3	17:00:23	EXPLAIN ANALYZE SELECT * FROM orders o INNER JOIN restaurants r ON o.restaurant_id = r.id WHERE o.c...	1 row(s) returned	0.016 sec / 0.00
4	17:01:08	CREATE INDEX idx_customer_created ON orders(customer_id, created_at)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.062 sec

- d) Re-run the same query and compare the new numbers to step (b).

The screenshot shows a database IDE with a SQL editor and an output window. The SQL editor contains the following code:

```
17
18
19 • EXPLAIN ANALYZE
20 SELECT *
21 FROM orders o
22 INNER JOIN restaurants r
23 ON o.restaurant_id = r.id
24 WHERE o.customer_id = 1
25 ORDER BY o.created_at DESC
26 LIMIT 5;
27
28
```

The output window shows the following actions and messages:

#	Time	Action	Message	Duration / Fetch
1	17:00:10	EXPLAIN ANALYZE SELECT * FROM orders o INNER JOIN restaurants r ON o.restaurant_id = r.id WHERE o.c...	Error Code: 1046. No database selected Select the default DB to be used by double-clicking its name in the SCH...	0.016 sec
2	17:00:18	USE tasks	0 row(s) affected	0.000 sec
3	17:00:23	EXPLAIN ANALYZE SELECT * FROM orders o INNER JOIN restaurants r ON o.restaurant_id = r.id WHERE o.c...	1 row(s) returned	0.016 sec / 0.000 s
4	17:01:08	CREATE INDEX idx_customer_created ON orders(customer_id, created_at)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.062 sec
5	17:02:49	EXPLAIN ANALYZE SELECT * FROM orders o INNER JOIN restaurants r ON o.restaurant_id = r.id WHERE o.c...	1 row(s) returned	0.000 sec / 0.000 s

e) Rewrite the query to select only the exact columns the profile page needs — no SELECT *.

The screenshot shows a MySQL IDE window with a query editor and a results panel. The query editor contains the following SQL code:

```

26
27 • EXPLAIN ANALYZE
28 SELECT
29     r.name,
30     o.total,
31     o.status,
32     o.created_at
33 FROM orders o
34 JOIN restaurants r
35 ON o.restaurant_id = r.id
36 WHERE o.customer_id = 1
37 ORDER BY o.created_at DESC
38 LIMIT 5;
39

```

The results panel shows the execution plan for the query. The first row of the plan is:

```

-> Limit: 5 row(s) (cost=2.1 rows=3) (actual time=2.15..2.25 rows=3 loops=1)  -> Nested loop inner join (cost=2.1 rows=3) (actual time=2.14..2.24 rows=3 loops=1)  -> Filter: (o.restaurant_id is not null) (cost=1.05 rows=3) (actual time=1.7...

```

The second row of the plan is:

```

EXPLAIN

```

The third row of the plan is:

```

-> Limit: 5 row(s) (cost=2.1 rows=3) (actual time=2.15..2.25 rows=3 loops=1)  -> Nested loop inner join (cost=2.1 rows=3) (actual time=2.14..2.24 rows=3 loops=1)  -> Filter: (o.restaurant_id is not null) (cost=1.05 rows=3) (actual time=1.7...

```

The fourth row of the plan is:

```

EXPLAIN ANALYZE SELECT * FROM orders o INNER JOIN restaurants r ON o.restaurant_id = r.id WHERE o.customer_id = 1 ORDER BY o.created_at DESC LIMIT 5;

```

The fifth row of the plan is:

```

EXPLAIN ANALYZE SELECT r.name, o.total, o.status, o.created_at FROM orders o JOIN restaurants r ON o.restaurant_id = r.id WHERE o.customer_id = 1 ORDER BY o.created_at DESC LIMIT 5;

```

The results panel also shows the output of the query, which is a table with 5 rows and 4 columns: name, total, status, and created_at.

f) Submit your before/after comparison, plus 2–3 sentences explaining why the index you chose helped.

→ Befor : When we use SELECT * it takes 3.21sec time anwhen we use particular column name it takes 2.15sec . The composite index on (customer_id, created_at) improves performance because MySQL can quickly locate all orders for the selected customer and retrieve them in the required order without scanning the entire table. This reduces the number of rows examined and decreases the query execution time, making the customer profile page load faster.